SatsPath v2 Server-to-Server (S2S) Wire Protocol Specification

Status: Draft / Normative Specification
Parent Epic: #46
Issue: #48
Depends On: docs/s2s/trust-model-v2.md

---

1. Overview & Protocol Philosophy

The SatsPath v2 S2S Wire Protocol defines the HTTP and cryptographic interface that clients, resolvers, and peer servers use to discover authority, resolve identifiers, verify cryptographic proofs, and synchronize transparency logs.

Core Principles

1. **Self-Contained Proof Envelopes**: A resolution response carries all cryptographic proofs necessary for a pure verifier to establish integrity, origin, and freshness without querying secondary endpoints.
2. **Deterministic Canonicalization**: All cryptographic commitments use strict domain separation (`SatsPath*V1` / `SatsPath*V2`) and canonical JSON (RFC 8785) formatting.
3. **Fail-Closed Verification**: Any missing mandatory field, unknown schema extension, malformed proof, or broken consistency chain terminates verification immediately.

---

2. Cryptographic Encodings & Domain Separation

All hashes are SHA-256 over domain-separated prefixes followed by RFC 8785 canonical bytes.

Domain Constant	Purpose
SatsPathNamespaceDescriptorV1	Namespace authority and endpoint metadata
SatsPathNameEventPayloadV1	Owner-signed name lifecycle action
SatsPathSignedNameEventV1	Merkle leaf commit over signed event
SatsPathTransparencyCheckpointV1	Operator signed log & state checkpoint
SatsPathWitnessCosignatureV1	Witness cosignature committing checkpoint
SatsPathCurrentStateEntryV1	Authenticated current-state map entry

---

3. Protocol Data Objects (Rust & JSON Schemas)

3.1. Namespace Descriptor (`NamespaceDescriptor`)

Discovered via DNS TXT or `/.well-known/satspath-authority`.

```rust
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq)]
pub struct NamespaceDescriptor {
    pub version: u16,                  // Must be 2
    pub domain: String,                // Canonical domain name (e.g. "example.com")
    pub log_id: String,                // Unique Log ID (hex SHA-256)
    pub authority_pubkey: String,      // Compressed secp256k1 hex (66 chars)
    pub endpoint_urls: Vec<String>,    // HTTPS endpoints (e.g. ["https://s2s.example.com"])
    pub witness_quorum: u8,            // Minimum witness count required (e.g. 2)
    pub witness_pubkeys: Vec<String>,  // Authorized witness public keys
```

```
    pub valid_from: i64,                // UNIX timestamp (seconds)
    pub expires_at: i64,                // UNIX timestamp (seconds)
    pub signature: String,              // Signature by authority_pubkey
}
```

3.2. Witness Cosignature (`WitnessCosignature`)

Guarantees global consistency and split-view prevention.

```
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq)]
pub struct WitnessCosignature {
    pub version: u16,
    pub witness_id: String,             // Witness identifier or URL
    pub witness_pubkey: String,         // Compressed secp256k1 hex
    pub checkpoint_hash: String,        // Hash of TransparencyCheckpoint
    pub tree_size: u64,                 // Committed tree size
    pub timestamp: i64,                 // Witness observation timestamp
    pub signature: String,              // Signature over checkpoint_hash + tree_size + timestamp
}
```

3.3. Resolution Envelope (`ResolutionEnvelope`)

The comprehensive proof payload returned on resolution.

```
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq, Eq)]
pub struct ResolutionEnvelope {
    pub version: u16,                               // Must be 2
    pub identifier: String,                         // Canonical identifier (e.g. "alice@example.com")
    pub namespace_descriptor: NamespaceDescriptor,  // Authority descriptor
    pub signed_profile: SignedPaymentProfile,       // Resolved payment profile
    pub name_events: Vec<NameEvent>,                // Monotonic event chain (0..N)
    pub inclusion_proof: MerkleInclusionProof,      // RFC 6962 leaf inclusion proof
    pub checkpoint: TransparencyCheckpoint,         // Log operator signed checkpoint
    pub consistency_proof: Option<MerkleConsistencyProof>, // Proof from client's pinned tree
    pub current_state_proof: Option<CurrentStateProof>, // Non-inclusion / current pointer proof
    pub witness_cosignatures: Vec<WitnessCosignature>,  // Cosignatures meeting quorum
    pub served_at: i64,                             // Response timestamp
}
```

---

4. HTTP Endpoints & API Surface

All endpoints use `Content-Type: application/vnd.satspath.v2+json; charset=utf-8`.

4.1. Authority Discovery

- **Endpoint**: `GET /.well-known/satspath-authority`
- **Query Parameters**: None
- **Response**: `200 OK` with `NamespaceDescriptor`
- **Caching**: `Cache-Control: public, max-age=3600, stale-while-revalidate=86400`

4.2. Proof-Carrying Identifier Resolution

- **Endpoint**: `GET /v2/resolve`
- **Query Parameters**:
  - `identifier` (required): Canonical format `user@domain.com` (lowercase, UTF-8).

  - `pinned_tree_size` (optional): Client's last observed tree size to request inline consistency proof.
  - `include_history` (optional): `true` / `false` (defaults to `true`).
- **Alternative**: `POST /v2/resolve` with JSON body `{"identifier": "...", "pinned_tree_size": ...}` to avoid URL proxy logging of private sub-identifiers.
- **Response**: `200 OK` with `ResolutionEnvelope`.
- **Errors**:
  - `404 Not Found`: Identifier does not exist (returns cryptographically signed non-inclusion proof).
  - `400 Bad Request`: Malformed identifier syntax.
  - `503 Service Unavailable`: Log synchronization in progress.

4.3. Checkpoint Retrieval

- **Endpoint**: `GET /v2/checkpoint`
- **Response**: `200 OK` with latest `TransparencyCheckpoint` and attached `witness_cosignatures`.
- **Caching**: `Cache-Control: public, max-age=15`

4.4. Consistency Proof Retrieval

- **Endpoint**: `GET /v2/proof/consistency`
- **Query Parameters**:
  - `old` (required): Previous tree size N_1.
  - `new` (required): Target tree size N_2 ($N_2 \geq N_1$).
- **Response**: `200 OK` with `MerkleConsistencyProof` (RFC 6962 format).

4.5. Monitor Entry Stream

- **Endpoint**: `GET /v2/entries`
- **Query Parameters**:
  - `from` (required): Start leaf index (inclusive, 0-indexed).
  - `to` (required): End leaf index (inclusive, max delta: 1000).
- **Response**: `200 OK` with JSON array of `NameEvent` leaves.

---

5. Pure Verification Algorithm (Normative)

A compliant SatsPath v2 verifier executes the following ordered sequence:

```
[1. Namespace Verification]
      Validate DNSSEC on _satspath.domain.com OR check WebPKI / Pin.
      Verify namespace_descriptor.signature against authority_pubkey.
      Check namespace_descriptor expiration: expires_at > now.

[2. Profile & Event Chain Verification]
      Verify signed_profile signature with profile.identity_pubkey.
      Compute profile_hash = H(SatsPathSignedPaymentProfileV1 || canonical_profile || signature).
      Verify name_events sequence (0..N) monotonic hash chain.
      Assert name_events[N].profile_hash == profile_hash.
      Assert name_events[N].identity_pubkey == signed_profile.identity_pubkey.

[3. Merkle Leaf & Checkpoint Inclusion]
      Compute leaf_hash = H(0x00 || H(SatsPathSignedNameEventV1 || canonical(name_events[N]))).
      Verify inclusion_proof connects leaf_hash to checkpoint.log_root at log_size.
      Verify checkpoint.operator_signature using checkpoint.operator_pubkey.

[4. Checkpoint Extension & Consistency]
```

```
      If client has pinned checkpoint (size S_old, root R_old):
           If checkpoint.log_size < S_old: FAIL(ERR_CHECKPOINT_ROLLBACK).
           If checkpoint.log_size == S_old: Assert checkpoint.log_root == R_old.
           If checkpoint.log_size > S_old: Verify consistency_proof(S_old, S_new, R_old, R_new).
      If operator_pubkey changed: Verify operator_rotation dual-signatures.

[5. Witness Quorum Verification]
      Filter witness_cosignatures matching namespace_descriptor.witness_pubkeys.
      Verify each witness signature over checkpoint_hash and tree_size.
      Assert valid_witness_count >= namespace_descriptor.witness_quorum.
```

---

6. Limits, Timeouts & Error Mapping

Parameter	Limit / Policy
Max Resolution Envelope Size	65,536 bytes (64 KiB)
Max History Events in Response	256 events (older history retrieved via `/v2/entries`)
Max S2S Request Timeout	5000 ms
Max Allowed Clock Skew	$\pm$300 seconds (5 minutes)
Allowed Compression	`gzip`, `zstd`, `br`

Terminal Error Mapping

```
HTTP 400 Bad Request           -> ERR_MALFORMED_ENVELOPE / ERR_PAYLOAD_TOO_LARGE
HTTP 404 Not Found             -> ERR_NON_INCLUSION_UNVERIFIED (or valid authenticated non-existence)
HTTP 422 Unprocessable Entity  -> ERR_BROKEN_HISTORY_CHAIN / ERR_INCLUSION_MISMATCH
HTTP 502 Bad Gateway           -> ERR_SERVER_UNAVAILABLE
HTTP 504 Gateway Timeout       -> ERR_SERVER_UNAVAILABLE
```